

Introduction of C++

C++ is a general-purpose, high-level, compiled, statically-typed programming language that supports both procedural-oriented and object-oriented programming paradigms. It was developed by Bjarne Stroustrup at AT&T Bell Laboratories, USA in 1979 and was publicly released in 1985.

It was developed as an extension of the C programming language by adding OOP features and several other advanced features. Initially it was called 'C with classes' but later it was renamed to 'C++' in 1984.

Versions of C++: -

There are some following major versions of C++: -

1998 - C++ 98
2003 - C++ 03
2011 - C++ 11
2014 - C++ 14
2017 - C++ 17
2020 - C++ 20
2023 - C++ 23

Features of C++

There are some following important features of C++: -

1. Simple
2. High-level
3. Case-sensitive
4. Compiler-based
5. object-oriented
6. Statically-typed
7. Powerful & fast
8. Machine-independent
9. Memory management
10. Rich library

1. Simple: - This programming language is very easy to learn and understand because its syntax is simple. And if we are familiar with C then it is very easy to learn C++.

2. High-level: - It is a high-level programming language, so it is very easy to deal with it. Because of high-level programming language is closely associated with human comprehensible English language.

3. Case-sensitive: - It is a case-sensitive programming language. It means C++ treats the uppercase characters and lowercase characters in different manner.

4. Compiler-based: - It is a compiler-based programming language. Compiler translates C++ source code in machine code directly. That's why it is a faster language than other programming languages like python, java, PHP, etc.
5. Object-oriented: - It is an Object-oriented programming language. It supports OOP concept such as class & object, inheritance, polymorphism, abstraction, and encapsulation.
6. Statically-typed: - It is a statically-typed programming language in which data type of a variable is decided at compile time. A statically-typed programming language is generally faster and performs better than a dynamically-typed programming language.
7. Powerful & fast: - It is a powerful and fast programming language compared to other programming languages like Python, Java, PHP, etc because it supports both low-level and high-level programming features.
8. Machine-independent: - It is a machine-independent programming language. It means we can execute C++ programs at different machines but at the same OS on which it is developed and compiled.
9. Memory management: - It supports dynamic memory allocation. It means we can free the allocated memory at any time. It helps to save the memory and improve program efficiency.
10. Rich library: - This programming language offers many different built-in functions. We can directly access them by including in our program which makes development easy.

Applications of C++

There are some following important types of applications which can be developed by using C++: -

- Operating systems
- Device drivers
- Window applications
- Web browsers
- Video games

Simple C++ program

Syntax,

```
#include<iostream>
using namespace std;

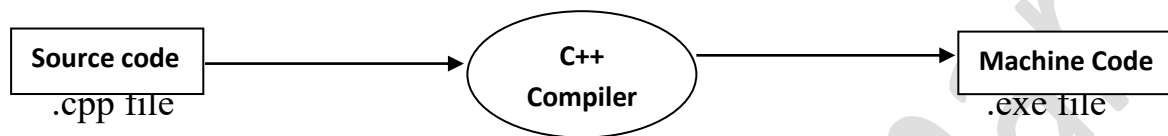
int main()
{
    // Code to be executed
    return 0;
}
```

Example,

```
#include<iostream>
using namespace std;
int main ( )
{
    cout<< "Hello C++";
    return 0;
}
```

📁 Save: - file_name.cpp

Compile & Run: -



IDE

IDE (Integrated Development Environment) is application software that provides platform to develop applications. It provides software developing tools such as editor, compiler, debugger, etc under a single GUI. This makes application development easier. There are some following types of popular IDE which are used for C++ programming: -

- Turbo C++
- Dev C++
- CodeBlocks
- VSCode
- Eclipse

Difference between C and C++

<u>C</u>	<u>C++</u>
• It is a middle-level programming language. • It is developed by Dennis Ritchie in 1972. • It supports procedural-oriented paradigm. • It is a subset of C++. • It does not support classes and objects. • It follows top-down approach. • It does not support access modifiers. • It does not support overloading and inheritance concept.	• It is a high-level programming language. • It is developed by Bjarne Stroustrup in 1979. • It supports both procedural-oriented and object-oriented paradigm. • It is a superset of C. • It supports classes and objects. • It follows bottom-up approach. • It supports access modifiers. • It supports overloading and inheritance concept.

Class & Object

A class is a user-defined data type that acts as a blueprint or template from which objects are created. A class is declared using the 'class' keyword. A class consists of data members and member functions. Memory is not allocated when a class is created.

Object-Oriented Programming (OOP) is a programming paradigm used to design programs using classes and objects. The four main characteristics (pillars) of OOP are, inheritance, polymorphism, abstraction, and encapsulation.

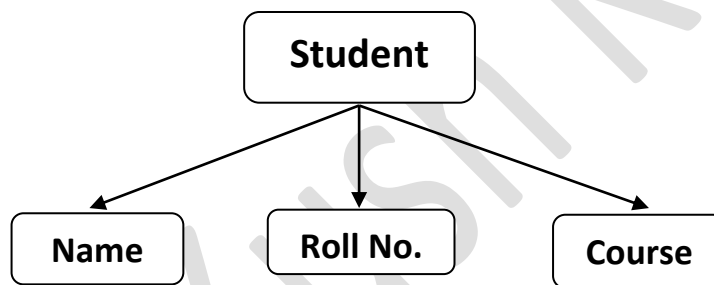
Syntax,

```
class class_name
{
    // Data members and member functions
};
```

An object is an instance of a class. Memory for data members is allocated only when an object of the class is created.

Syntax,

```
class_name object_name;
```



For example,

```
#include<iostream>
#include<string>

using namespace std;

class Student
{
public:
    string name;
    int roll;
    string course;

    void show()
    {
        cout<<"Name : " <<name <<endl;
        cout<<"Roll No. : " <<roll <<endl;
        cout<<"Course : " <<course <<endl;
    }
};
```

```

int main( )
{
    Student s1;

    s1.name ="Lav Kush";
    s1.roll =16;
    s1.course = "BCA";

    s1.show( );

    return 0;
}

```

There are following differences between POP (Procedural-Oriented Programming) and OOP (Object-Oriented Programming): -

<u>POP</u>	<u>OOP</u>
1. In POP, the program is divided into smaller parts called functions.	1. In OOP, the program is divided into smaller parts called objects.
2. It follows top-down approach.	2. It follows bottom-up approach.
3. Creation of large, complex, and scalable programs is difficult in POP.	3. Creation of large, complex, and scalable programs is easier in OOP.
4. It takes less memory compared to OOP.	4. It takes more memory compared to POP.
5. It gives more importance to procedures rather than data.	5. It gives more importance to data rather than procedures.
6. It is less secure compared to OOP because data is mostly global.	6. It provides better data security through data hiding and encapsulation.
7. Example of POP languages are C, BASIC, FORTRAN, etc.	7. Examples of OOP languages are C++, JAVA, Python, etc.

Access Modifiers

Access modifiers are keywords used to control the accessibility of class members. They decide how and from where members of a class can be accessed. They are also called access specifiers.

There are the following three types of access modifiers available in C++: -

1. public
2. protected
3. private

1. public: - The 'public' keyword is used to declare class members as public. Public members are accessible anywhere in the program either within the same class, in derived classes or from outside the class.

2. protected: - The 'protected' keyword is used to declare class members as protected. Protected members are accessible within the same class and in derived classes. These are not accessible from outside the class.

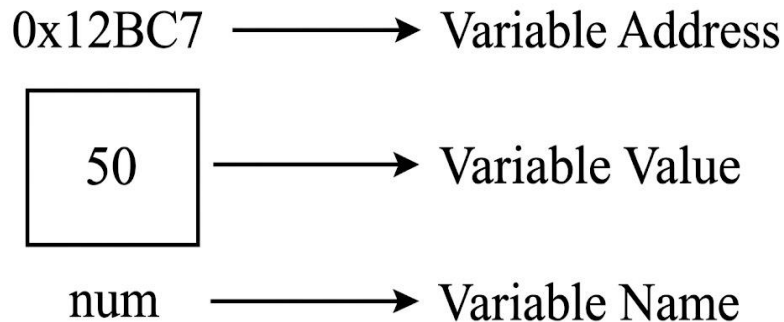
3. private: - The 'private' keyword is used to declare class members as private. Private members are accessible only within the same class by its member functions. In C++ class members are private by default.

NOTE: - A friend class and friend function can also access private and protected member of a class.

Access Modifier	Within Same Class	Within Derived Class	Outside of the Class
public	Yes	Yes	Yes
protected	Yes	Yes	No
private	Yes	No	No

Variable

A variable is a symbolic name given to a memory location that is used to store data during the execution of a program. The value stored in a variable can be accessed and changed during program execution.



Rules for naming variables

- A name of a variable can be composed of alphabets (A-Z, a-z), numbers (0-9) and underscore (_). But it must start with an alphabet (A-Z, a-z) or underscore (_).
- Whitespace is not allowed in a variable name.
- Keywords cannot be used as variable name.
- A variable name is case-sensitive.
- A variable must be declared before it is used.

Declaration,
`data_type variable_name;`

Initialization,
`variable_name = value;`

Declaration & Initialization,
`data_type variable_name = value;`

In C++11, curly braces { } was introduced as another way to initialize variables. It helps to prevent narrowing conversions. It is known as uniform initialization or brace initialization.

Declaration & Initialization,
`data_type variable_name{ };`
‘Or’
`data_type variable_name{value};`
‘Or’
`data_type variable_name = {value};`

Auto Variable

A variable that is declared using the '**auto**' keyword is called an auto variable. It was introduced in C++11. The compiler automatically determines the data type of a variable based on the value assigned to it during initialization. An auto variable must be initialized at the time of declaration.

Syntax,

```
auto variable_name = value;
```

Example,

```
auto a = 10;
```

There are following types of variables available in C++: -

1. Local variable
2. Global variable
3. Static variable

1. **Local variable:** - A variable which is declared inside the function body is called local variable. We can access these variables only within the function. Local variables are created when function execution starts and destroyed when the function execution ends.

NOTE: - The default value of local variables is garbage value.

2. **Global variable:** - A variable which is declared outside of all functions and classes is called global variable. It can be used anywhere in the program. Global variables are created when the program starts and destroyed when the program terminates.

NOTE: - The default value of global variables is zero (0).

3. **Static variable:** - A variable which is declared with the 'static' keyword is called static variable. It can be declared either inside of the function body or outside of the function body. Static variables are created when program starts and destroyed when the program terminates. It retains its value on every function call.

NOTE: - The default value of static variables is zero (0).

For example,

```
#include <iostream>
using namespace std;

int a = 10; // Global variable
static int b = 20; // Global static variable

int main(void)
{
    int c = 30; // Local variable
```

```
static int d = 20; //Local static variable

cout<<a<<endl;
cout<<b<<endl;
cout<<c<<endl;
cout<<d<<endl;

return 0;
}
```

Constant

A constant is a type of variable whose value can't be modified once it is initialized. If we try to change its value, the compiler throws an error.

To define constant 'const' keyword is used in C++. Its value can be assigned either at compile-time or at runtime.

Syntax,

```
const data_type variable_name = initial value;
```

Example,

```
const float PI = 3.14f;
```

In C++11 'constexpr' keyword was introduced to define a constant. It guarantees that the value is evaluated at compile-time, which improves runtime performance.

Syntax,

```
constexpr data_type variable_name = initial value;
```

Example,

```
constexpr float PI = 3.14f;
```

Data types

Data types are used to declare the type and size of variables. C++ is a statically typed programming language. It means that the type of a variable is determined at compile-time.

There are following types of data types available in C++ programming language: -

1. Primitive data types
2. Derived data types
3. User-defined types

1. Primitive data types: - Primitive data types are data types that are predefined in C++. They are also known as built-in data types. They can be used directly in a program. There are following primitive data types available in C++: -

- **Integer:** - Integer data type is used to store numeric value. The '**int**' keyword is used to declare an integer variable. It takes 4 bytes of memory.

↳ Range: - (-2147483648 to 2147483647)

Syntax,

`int variable_name = value;`

Example,

`int num = 12354;`

- **Float:** - Float data type is used to store decimal value. It provides approximately 6–7 significant decimal digits. The '**float**' keyword is used to declare a float variable. It takes 4 bytes of memory.

↳ Range: - (-3.4 e -38 to +3.4 e+38)

Syntax,

`float variable_name = valuef;`

Example,

`float f1 = 245.250f;`

- **Double:** - Double data type is also used to store decimal number. It can store up to 15–16 significant decimal digits. The '**double**' keyword is used to declare a double variable. It takes 8 bytes of memory.

↳ Range: - (-1.7 e -308 to 1.7 e+308)

Syntax,

`double variable_name = value;`

Example,

`double d1 = 548998.7894555;`

- **Character:** - Character data type is used to store a single character. The '**char**' keyword is used to declare a character variable. It takes 1 byte of memory.

↳ Range: - (-128 to 127)

Syntax,

```
char variable_name = 'Value';
```

Example,

```
char ch = 'A';
```

- **Boolean:** - Boolean data type is used to store a logical value which can be either true (1) or false (0). The '**bool**' keyword is used to declare a boolean variable. It takes 1 byte of memory.

Syntax,

```
bool variable_name = value;
```

Example,

```
bool b1 = true;
```

- **Void:** - Void means nothing. It cannot hold any value. Void data type is used for the function that does not return a value.

NOTE: - We can't declare a variable of void data type.

2. Derived data types: - Derived data types are data types that are derived from the built-in data types.

There are following types of derived data types available in C++ programming language: -

- Function
- Array
- Pointer
- Reference

3. User-defined types: - User-defined data types are data types that are defined by the user itself.

There are following types of user-defined data types available in C++ programming language: -

- Class
- Structure
- Union
- Enumeration
- Typedef

NOTE: - string is an object of the std::string class used to store a sequence of characters.

Syntax,

```
string variable_name = "value";
```

Example,

```
string name = "Hello Lav";
```

Data type modifiers

Data type modifiers are keywords in C++ programming language that are used to modify the default behaviours of built-in data types.

We can categorize data type modifiers into following two types: -

1. Size
2. Sign

1. Size: - It is used to modify the default size of built-in data types. There are following two types of size data type modifiers: -

- i. short
- ii. long

i. short: - It is used to reduce the default size of built-in data type. It can be only used with int data type. It takes 2 bytes of memory.

↳ Range: - (-32768 to 32767)

Syntax,

```
short int variable_name = value;  
‘Or’  
short variable_name = value;
```

Example,

```
short int num = 12354;  
‘Or’  
short num = 12354;
```

ii. long: - It is used to increase the default size of built-in data types. It can be only used with integer and double data type.

(a) long long int: - It is used to store numeric value greater than int data type. It takes 8 bytes of memory.

↳ Range: - (-9223372036854775808 to 9223372036854775807)

Syntax,

```
long long int variable_name = value;  
‘Or’  
long long variable_name = value;
```

Example,

```
long long int num = 9608375686;  
‘Or’  
long long num = 9608375686;
```

(b) long double: - It stores real value greater than float data type. It can store up to 18 decimal point numbers. It takes 12 bytes of memory.

↳ Range: -(3.4 e -4932 to 1.1 e+4932)

Syntax,

```
long double variable_name = value;
```

Example,

```
long double num = 86756754.4758626565;
```

2. Sign: - It is used to modify the default sign of built-in data types. There are following two types of sign data type modifiers: -

i. signed

ii. unsigned

i. signed: - It is default sign modifier for int and char data type if no modifier is specified. It can store both negative and positive values.

(a) signed char: - It is used to store both negative and positive character value. It takes 1 bytes of memory.

↳ Range: - (-128 to 127)

Syntax,

signed char variable_name = 'Value';

'Or'

char variable_name = 'Value';

Example,

signed char ch = 'L';

'Or'

char ch = 'L';

(b) signed int: - It is used to store both negative and positive numeric value. It takes 4 bytes of memory.

↳ Range: - (-2147483648 to 2147483647)

Syntax,

signed int variable_name = Value;

'Or'

int variable_name = Value;

Example,

signed int i = -68548;

'Or'

int i = -68548;

ii. unsigned: - The unsigned data type modifier is used to store only non-negative values. It can be only used with int and char data types.

(a) unsigned char: - It is used to store non-negative character values. It takes 1 bytes of memory.

↳ Range: - (0 to 255)

Syntax,

unsigned char variable_name = 'value';

Example,

unsigned char uc = 'L';

(b) unsigned int: - It is used to store only non-negative integer values. It takes 4 bytes of memory.

↳ Range: - (0 to 4294967295)

Syntax,

unsigned int variable_name = value;

Example,

unsigned int ui = 45785444;

Function

A function is a block of code or a group of statements that is used to perform a specific task. It provides code reusability feature to C program and helps in decreasing redundancy. It executes only when it is called.

Syntax: -

```
return_type function_name(parameters_list)
{
    //line of code
}
```

There are the following two types of functions in C programming language: -

1. Pre-defined functions
2. User-defined functions

1. Pre-defined functions: - Pre-defined functions also known as library functions are the functions that are already defined in C standard library header files.

For example,

`sqrt( ), pow( ), strcmp( ), strlen( ), exit( ), etc.`

2. User-defined functions: - User-defined functions are the functions that are defined by the programmer to perform a specific task.

There are the following types of user defined functions: -

1. No Arguments and No Return Value
2. No Arguments but Return Value
3. Arguments but No Return Value
4. Arguments and Return Value

1. No Argument and No Return Value: - In this type of function, no values are passed during the function call, and the function does not return any value to the caller. It simply performs its task and displays the results.

function definition: -

```
void function_name ( )
{
    //block of code
}
```

function call: -

```
function_name ( );
```

2. No Argument but Return Value: - In this type of function, no values are passed during the function call, but the function returns a value to the caller. It performs its tasks and sends the result back to the caller.

function definition: -

```
return_type function_name()  
{  
    //block of code  
    return value;  
}
```

function calling: -

```
variable_name = function_name( );
```

3. Arguments but No Return Value: - In this type of function, values are passed during the function call, which is called arguments or actual parameters, but the function does not return any value. It performs a task using the given data and displays the result.

function definition: -

```
void function_name(parameters / formal parameters)  
{  
    //block of code  
}
```

function calling: -

```
function_name(arguments / actual parameters);
```

4. Argument and Return Value: - In this type of function, values are passed during the function call which is called arguments or actual parameters, and the function returns a value back to the caller.

function definition: -

```
return_type function_name(parameters / formal parameters)  
{  
    //block of code  
    return value;  
}
```

function calling: -

```
variable_name = function_name(arguments / actual parameters);
```

Function calling

Function calling is the process of invoking a function by using its name along with arguments. There are two types of function calling in C: -

1. Pass by value
2. Pass by reference

1. Pass by value: - In pass by value, a copy of the argument's value is passed to the function parameter. It does not change the original value. Any changes made to the parameter inside the function have no effect on the original value. By default, C++ uses call by value.

Syntax,

```
void function_name(data_type variable_name)
{
    //block of code
}
```

Calling,

```
function_name(value/variable);
```

2. Pass by reference: - In pass by reference, a reference to the original value is passed to the function. If we make any changes inside the function, it also makes changes to the original value.

Syntax,

```
void function_name(data_type &variable_name)
{
    //block of code
}
```

Calling,

```
function_name(variable);
```

Inline function

An inline function is a function that is declared with the inline keyword. It requests to the compiler to replace the function call with the actual function code. This may reduce function call overhead and improve program performance.

Note: - The inline keyword is only a request, not a guarantee, compiler may ignore it.

Syntax,

```
inline return_type function_name(parameters)
{
    // block of code
}
```

Example,

```
#include <iostream>
using namespace std;

inline int square(int num)
{
    return num * num;
}

int main()
{
    int num, result;

    cout << "Enter a number: ";
    cin >> num;

    result = square(num);

    cout << "Square of " << num << " is " << result;

    return 0;
}
```

Recursion

Recursion is a programming technique in which a function calls itself either directly or indirectly until a base condition is reached to solve a problem. A function that calls itself either directly or indirectly is called a recursive function.

Determining how many times a recursive function executes and analysing its time complexity is known as recursive analysis. There are a few important parts that need to be considered during recursion: -

1. Base case: - It is the condition that stops the recursion. Without a base case, the function keeps calling itself indefinitely.
2. Recursive case: - In this case function calls itself with a smaller subproblem.
3. Recursive function call: - It is the statement inside the function that calls the same function again.

Example,

```
#include <iostream>
using namespace std;

int factorial(int num)
{
    if (num == 0 || num == 1)
    {
        return 1;
    }
    else
    {
        return num * factorial(num - 1);
    }
}

int main()
{
    int num;

    cout << "Enter a number to find the factorial: ";
    cin >> num;

    cout << "Factorial: " << factorial(num);

    return 0;
}
```